

How Big Tech Runs Tech Projects and the Curious Absence of Scrum

A survey of how tech projects run across the industry highlights Scrum being absent from Big Tech.



GERGELY OROSZ

SEP 21, 2021



279



5



3

Share



Welcome to another issue of **The Pragmatic Engineer** where I cover topics relevant for engineering managers and senior engineers at high-growth startups and big tech.

If you're reading this but haven't yet signed up, join over 100,000 others and get The Pragmatic Engineer delivered to your inbox every week by subscribing here 📌

Question: I moved to a large tech company from a startup. I want to help my team execute well, but I'm aware the environment is different. What project management best practices have you seen across the industry, and what advice would you have on which approach to choose?

Project management is a topic most people have strong opinions on, and I'm no exception. To answer this question, I pulled in help from across the industry as well. In this issue we'll cover:

- **Project management approaches across the industry.** An overview of a survey with over 100 companies represented, plus key takeaways.
- **Project management at Big Tech.** How are these done? How does the organizational setup of Big Tech influence how projects are executed?
- **The lack of Scrum at Big Tech.** Why is the popular framework missing from most of Big Tech, and are there takeaways for companies operating outside this model?
- **How should you run projects in your team?** I'll share my personal take.

Before we jump in, here's a personal story about why it's sometimes hard to put a finger on how important project management approaches are.

Skype, Scrum, and the Reminder of What Matters

When I joined Skype in 2012, the company had gone all-in on Scrum. All engineers and product people were sent to best-in-class Scrum training, facilitated by one of the Agile manifesto's founders. Skype went all-in on Scrum, moving all teams to this methodology over a few quarters.

And the move to Scrum was seen as a success at Skype. We went from shipping the flagship Windows app once-a-quarter at best, to monthly shipping. Most teams delivered something every 2-4 weeks. Teams rotated Scrum Master roles, agile coaches dropped in to give feedback on the teams, and Microsoft – which had just acquired Skype – was interested in taking inspiration from the speedup in delivery.

However, while Skype moved over to Scrum, a competitor was executing ruthlessly: Whatsapp. Though a much smaller organization, Whatsapp chipped away market share month after month, becoming the leading communications platform.

Unlike Skype, Whatsapp never bothered with a framework like Scrum. Early employees shared how they never even muttered the word and deliberately ignored all heavyweight processes. Whatsapp out-executed Skype, built a more reliable messaging experience than Skype, and ultimately won the battle of messaging and communication apps.

The success of companies and project management approaches is not always correlated and this story is a reminder of this. I'm not saying how you run projects is not important: it is. But there are other things that might have a greater impact on outcomes, such as focus, leadership approaches, how people work even without a process, and so on.

Project management is a piece in a complex and ever-changing puzzle. However, it is not – and should not be – an end goal, only an enabler to reach that goal quicker.

Project Management Approaches in the Industry

How do companies run projects? I ran a survey with over 100 responses. The results are interesting.

The summary of how companies manage projects is “it depends”. And this should not be very surprising. A newly founded startup with five people will see success in different ways from a 1,000-person, slowly growing non-tech company. Even within the group of large non-tech companies, some experiment with novel approaches, while others stick to doing the same thing that has been working well enough for years.

I segregated the companies based on whether tech was at their core, and their funding model.

Methodologies used by companies in this survey were:

- *No “formal” methodology*: common for public and venture-funded tech companies.
- *Plan, build, ship*: common for public and venture-funded tech companies.
- *Scrum*: common for large, non-tech companies, non-venture funded companies and consultancies.
- *Kanban*: mentioned across all companies.
- *SAFe (Scaled Agile Framework)*: mentioned with large, non tech companies and a non-venture funded company.
- *Shape Up*: mentioned for a few venture-funded companies.

Is There a “Central” Methodology?	Most Common Project Management Methodologies (In Order)	Who Typically Leads Engineering Projects?
Big Tech & Public Tech Companies		
- Teams can choose how they work (common) - Suggested methodology, but teams can choose (less frequent)	1. Plan->Build (iterate)->Ship 2. No "formal" methodology	- Tech lead - An engineer on the team
Venture-funded scaleups (Series B & above)		
- Suggested methodology, but teams can choose (common) - Teams expected to follow specific a methodology (less frequent)	1. Plan->Build (iterate)->Ship 2. No "formal" methodology 3. Kanban 4. Scrum	- Tech lead - An engineer on the team
Venture-funded startups (up to Series A)		
- Teams expected to follow specific a methodology (common) - Suggested methodology, but teams can choose (less frequent)	1. Plan->Build (iterate)->Ship 2. Scrum 3. Kanban	- An engineer on the team - Product manager
Non-venture funded tech companies		
- Teams expected to follow specific a methodology (mostly) - Suggested methodology, but teams can choose (less frequent)	1. Scrum 2. Others (Kanban, SAFe, Scaled Agile)	- Tech lead - Dedicated project manager - An engineer on the team
Large, non-tech companies		
- Teams expected to follow specific a methodology (mostly) - Suggested methodology, but teams can choose (rarely)	1. Scrum 2. SAFe 3. Others (Plan->build->ship, Kanban)	It varies: - A dedicated project manager - Scrum master - Product manager/owner - Tech lead
Consultancies		
- Teams expected to follow specific a methodology (mostly) - Suggested methodology, but teams can choose (rarely)	1. Scrum 2. No “formal” methodology	A dedicated project manager

newsletter.pragmaticengineer.com

How do companies run projects? An overview of the survey results.

The survey revealed a few interesting findings, some of which related to the question:
“On a scale of 1 (not satisfied) to 5 (very satisfied) how satisfied/happy are you with the

current project management methodology?"

Insights that are worth thinking about are below. I advise to treat these carefully, given the survey is non-representative in sample size. Nevertheless, they are observations I can confirm.

- **Teams with dedicated project managers** typically recorded lower satisfaction ratings at public or venture-funded tech companies. However, at non-venture funded companies and consultancies, several respondents were very happy with project management, and called out these people as a reason for their satisfaction.
- **Teams being allowed to choose their own way of working** was more common at public tech companies and venture-funded scaleups. Large, non-tech companies and smaller, non-venture-funded companies were more likely to mandate the same approach for all teams within the company.
- **Team autonomy and high satisfaction seemed to be correlated.** Many people rating their satisfaction as 4 or 5 mentioned autonomy, freedom and flexibility, and the putting of quality first at the team level, as a positive.
- **Teams struggling often had little to do with the methodologies.** People mentioned lack of vision, good engineers leaving, lack of transparency or poor tooling as reasons why things went badly. For these teams, no change of methodology would help because the issues ran deeper.
- **JIRA has been mentioned mostly with negative associations:** all [13 mentions of JIRA](#) were in this setting. Being able to get things done without working much with JIRA was mentioned as a positive. Additionally, a recently IPOed, high-growth tech company moved over to JIRA and ran a survey among engineers. It measured a Net Promoter Score (NPS) of -83. This is staggeringly low, and means that 83% of engineers would advise against JIRA. As a counterpoint, an engineering leader at a public tech company with stable growth shared that their organization heavily relies on JIRA, using it as a knowledge engine. In their case, JIRA works well for large-scale coordination, after the organization fully adopted it.

Project management approaches that do not work well share a few characteristics, according to respondents who left a rating of a 1 or a 2:

- **Engineers not involved in estimations** that the team then committed to, is a frequent pain point. In my view, it's one of the easiest ways to demotivate engineers – to the point of some leaving – and also to get a false sense of what a team can achieve.
- **Requirements changing, even with dedicated project managers**, sits poorly with engineers. While there are teams where requirements change a lot, on these teams it's typically the engineers who run the projects, and can deal with them. However, when a dedicated project manager is unable to shield the team from requirements changing, respondents rated this approach as poor.
- **Teams with no autonomy to change a failing project management approach also recorded low satisfaction.** These kinds of responses were pronounced at companies where all teams were expected to follow the same methodology. It's an example of directive leadership and while this approach can work well for roles where there is little creativity needed, it is usually a poor way to build high-performing software engineering teams. When teams can iterate *together* and change their processes on their own terms, satisfaction and productivity go up.

[Browse all anonymized responses here.](#)

How Big Tech Runs Projects

Big Tech differ in how they approach executing tech projects, compared to the rest of the industry. I gathered data by talking with people at well-known publicly traded tech companies. Here is how they typically get things done:

Company	Is There a “Central” Methodology?	What Project Management “Methodology” Is Typically* Used for Engineering Projects?	Who Typically Leads Engineering Projects?
Amazon	No, teams can choose	Plan (6-pager)->Build (iterate)->Ship	Tech lead
Apple	No, teams can choose	Plan->Build (iterate)->Ship	Tech lead
Datadog	No, teams can choose	Plan (RFC)->Build (iterate)->Ship	Tech lead or an engineer
Facebook	No, teams can choose	Plan->Build (iterate)->Ship	Tech lead or an engineer
Google	No, teams can choose	Plan (Design Doc)->Build (iterate)->Ship	Tech lead or an engineer
Netflix	No, teams can choose	Plan->Build (iterate)->Ship	Tech lead or an engineer
Shopify	No, teams can choose	GSD (Get Shit Done, 6-week cycles)	Tech lead or an engineer
Spotify	No, teams can choose	Plan->Build (iterate)->Ship	Tech lead or an engineer
Uber	No, teams can choose	Plan (ERD)->Build (iterate)->Ship	Tech lead or an engineer

newsletter.pragmaticengineer.com

How Big Tech runs engineering projects. Typically used methodology: as each team can choose how they work, the methodologies used per engineering team can vary, even on a project basis.

Big Tech shares several characteristics in how engineers execute on projects:

- **Engineers lead** most projects: either a tech lead, or an engineer on the team taking the lead.
- **Teams are free to choose** the project management methodology they use. Many teams go with an RFC-like planning process, iterate on building, and ship all within a few weeks. Other teams use more Kanban-like processes, where they work on the highest priority items.
- **There are no dedicated project managers** for team-level projects. Most of these companies have Technical Program Managers (TPMs) who step in for large projects involving multiple teams, or run across organizations. The ratio of TPMs to engineers was around 1:50 at Uber.
- **Project management artifacts and processes** vary between teams even in the same organization. Most teams have a project backlog, do standups at intervals as the team sees fit, and retrospectives every now and then.

- **First-class developer tooling is a given** in these places, and plays a large role in short iteration cycles. Many teams work on main branches, get quick feedback from CI/CD systems and can immediately share functionality which they are working on with other team members.

For people who have worked in Big Tech, much of this is familiar. However, if you were to try to copy this same approach in a more traditional company, it would likely fail. This is because the organizational structure of Big Tech greatly impacts how teams can – and do – execute.

Big Tech Organizational Structure that Impacts Projects

In order to understand how Big Tech manages projects, let's take a step back and look at the environment most of these companies operate in, which I dive deeper into, in the article [What Silicon Valley "Gets" about Software Engineers that Traditional Companies Do Not](#).

1. **Autonomy for software engineers and teams.** The expectation of developers at traditional companies is to complete assigned work. At SV-like companies, it's to solve problems that the business has. This is a huge difference. It impacts the day-to-day life of any engineer.
2. **Curious problem solvers, not mindless resources.** A motivated engineer easily makes multiple times the impact of a "factory worker" who only does what they're told. For organizations with a factory worker attitude, this approach will bias towards more heavyweight project management approaches that leave little room for interpretation, on purpose.
3. **Internal data, code, and documentation transparency.** Employees – and not just engineers – often have access to real time business metrics and data sources, to write their own queries and create custom reports.
4. **Exposure to the business and to business metrics.** Engineers are encouraged to interact with the rest of the business and build relationships with non-engineers. In contrast, traditional companies often make it impossible for developers to interact with the rest of the business.

5. **Engineer-to-engineer comms over triangular-communication.** Traditional companies will encourage hierarchical communication that slows down information flow, and results in slower decisions.
6. **Investing in a less frustrating developer experience.** Companies that care about engineers solving problems quickly [set up various platform teams](#), which reduce the developer experience churn.
7. **Higher pay, justified by higher leverage.** Companies that leverage engineers well, have no trouble paying close to the top of the market, or above it.
8. **Caliber of the talent hired.** These companies hire highly competent and highly motivated people, thanks to the combination of all the above. They have a large pool to choose from, as they are known for generous compensation packages, and strong career growth opportunities.

Empowered and autonomous teams are the building blocks of all these companies. They are also the key differentiator between many companies in the tech industry.

While not all teams within Big Tech will be fully empowered, these organizations – along with progressive thinking startups – are closest to implementing ways that ensure teams are as empowered as possible.

Teams with clear ownership is another building block of Big Tech. Each team of 5-15 people has a clear vision and mission, and the skills and autonomy to execute on it. The mission could be “Build a world-class booking experience for seniors” for a product team on a hospitality product, or “Empower teams to integrate rating experiences with close to zero effort” for a [product platform team](#).

Product-focused teams are typically composed of cross-functional team members like backend, frontend and/or mobile engineers, with product managers, data scientists and designers often also on the team. [Platform-focused teams](#) tend to not be cross-functional or tied to a specific domain.

Note that even though engineers have high-level specialization, in Big Tech most experienced software engineers are expected to be able to pick a broad range of engineering work, and the interview process also reflects this generalist approach.

Product Managers: Yes, Project Managers: No

Another curious difference between Big Tech and everyone else is the role of Product Managers, and the lack of Project Managers or Product Owners who are dedicated to teams.

The role of product managers at these companies is defining the strategy at the team – the “why” – and the steps to execute this strategy – the “how”. As Facebook product manager Will Lawrence [phrases this](#):

*“The role of a product manager is to figure out **what game we're playing** and **how we're going to play it**. Strategy is choosing the game we're playing. It's finding worthwhile areas to invest in and creating a compelling vision for how we can succeed in this game. (...) Execution is how we play the game. It's the day-to-day processes, decisions and actions we take to make progress towards our mission.”*

Product Managers ensure that the team keeps working on the right thing. This means working with the business, with data science and with design to build a roadmap, create plans, prioritize work and escalate where needed. At large companies, this itself is a full-time job already.

In many cases, Product Managers do not own project management at Big Tech.

The team is responsible for the execution, and the team lead – usually the Engineering Manager – is responsible for making this happen.

With empowered and autonomous teams, managing a project is rarely a top down exercise. Each team will vary, but I've found great success in rotating the project lead role among engineers, helping everyone on the team grow their leadership muscle.

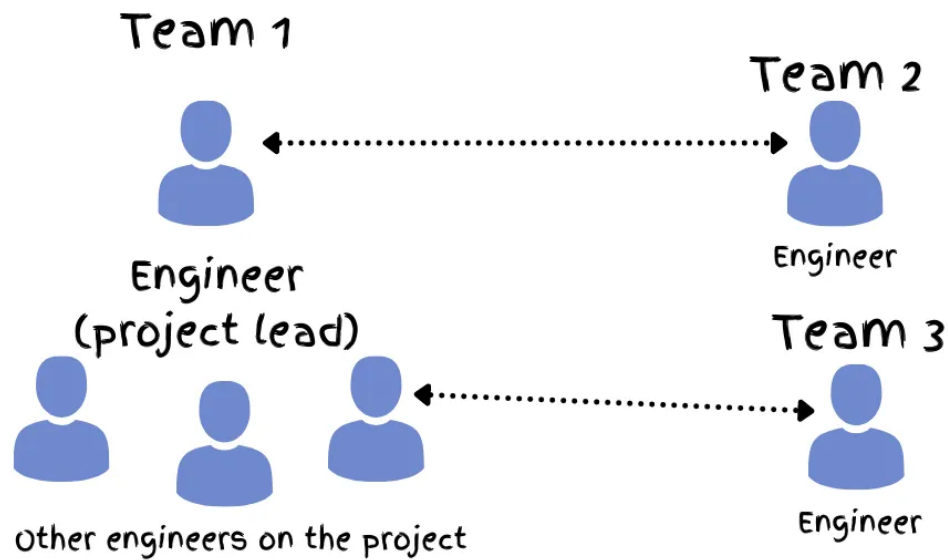
The lack of dedicated project managers raises several questions. Are engineers overloaded with project management? Is managing projects a good use of engineering time? All of these are possible, and here is my take.

- **For team-level projects**, not having a dedicated project manager ends up simplifying processes, and strengthening personal relationships. Engineering project leads will add as little process as they can, as this is in their interest. When

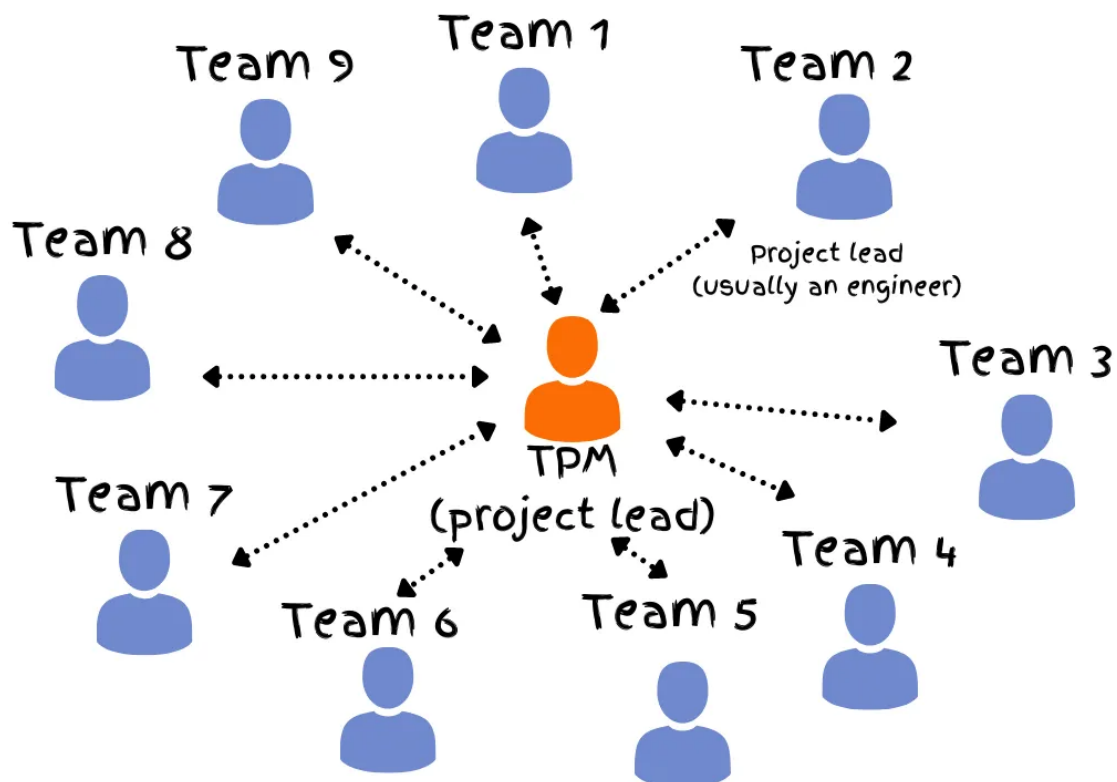
collaborating with other teams – also without project managers – they will also build relationships for other engineers leading projects, or owning products. This kind of engineer-to-engineer communication is more efficient than if it went through multiple project managers as well.

- **For complex projects** that span several teams across different offices and time zones, leading such a project is a full-time job for an engineer. Big Tech pulls in Technical Program Managers (TPMs) to manage these complex, and often strategic projects, taking the load off engineers.
- **Dedicated Program Managers** or Project Managers still exist within Big Tech. They are typically tied to external-facing commitments and customers – such as a Program Manager for the Apple partnership – or to long-running initiatives, like a compliance program. Similar to how TPMs are not allocated to a single engineering team, these Program Managers or Product Managers also don't tend to have an engineering team, but they work across many teams instead.

Simple projects



Complex projects



newsletter.pragmaticengineer.com

How large tech companies run projects. Simple projects are typically led by engineers. Complex projects by Technical Program Managers.

Dedicated Program Managers might also run longer-running initiatives.

Product-Focused Environments and Dropping Scrum

I witnessed first-hand how a Scrum team dropped Scrum altogether when working at Skype. When I joined the Skype for Web team, we initially did two-week sprints, and followed the usual Scrum processes. We also had a split of software engineers and QA engineers. However, our shipping pace was only every two weeks, but we wanted to ship more frequently.

The first thing we did was make QA part of engineering. In the “old world”, an engineer would finish their work, check into their branch, update a ticket, and let the QA know it’s ready for review. The QA would take this ticket a day or two later, review it, and reopen the ticket if they found issues. This was a long delay.

We made a quiet, unofficial, change where all SDETs built production software as well, and all software engineers became responsible for testing their own code. Now we no longer had to wait days for feedback before shipping code to production. However, the bi-weekly sprints and the numerous Scrum rituals became the next problem.

Scrum got in the way of shipping on a daily basis. The whole idea of Scrum revolves around Sprints, of committing to tasks at the beginning of the sprint, working on these during the sprint, and demoing what we did at the end.

The process felt unnatural and like it had been forced on a fast-moving web team. We soon moved to a more fluid way of working, taking the Kanban approach. We stopped caring about sprints, and dropped most rituals that come with Scrum. We just cared about knowing what we’re working on now, and what it was we’d get done next.

Infrastructure and developer tooling removed the need for many Scrum rituals.

Scrum rituals like demoing to the Product Owner, signing off the work and then shipping it, assumes a few things:

- That the Product Owner is the one who can with certainty validate the work as done to spec.
- That the Product Owner wrote the spec- because they are validating it.

- That the work is not being shipped to production before the signoff is done.

However, in our environment, these assumptions were often flawed. All code we wrote was tested with unit tests and, where needed, with integration and end-to-end tests. We shipped our code behind feature flags, and validated them with staged rollouts, starting with a rollout to the team. A lot of the “specs” – or tickets – were also written by engineers, who could then validate if they worked as expected. CI/CD, feature flags and experimentation tooling allowed us to have faster feedback cycles than relying on a Product Owner.

Much of Big Tech has recognized how first-class infrastructure and developer tooling make a big difference to the productivity for engineering teams. This is why 30-40% of engineering often works on platform teams and is [why Uber invested heavily in platform teams](#). With first-class infrastructure and platforms ready to use, teams can focus on their core work goals, over figuring out how to set up infrastructure, or how to make a service compliant.

All members of the team were very clear on what we were building. Our end goal was to ship Skype’s web functionality. This approach was made up of several sub-projects, but the big picture was clear to the team. The Product Managers helped set the high-level strategy, and us engineers picked up the work to be done, and ran with it. We were empowered enough to chip away parts of Scrum that got in our way. After a while, what was left did not represent Scrum at all.

Beyond Scrum

When talking to engineers at Facebook, Whatsapp, Google, Netflix and similar organizations, most of them have never used Scrum. Why? It’s because of a few things:

- **Competent, autonomous people need less structure** to produce reliable, high-quality output. Big Tech is able to attract, afford and hire these people.
- **Leveraging competent teams comes through giving them freedom** to choose how to operate. This is true for most types of engineering, and there’s good reason why the [Skunk Works](#) model of autonomy with reduced bureaucracy, is what many high-performing teams with high-caliber people end up following.

Scaling an engineering organization goes well beyond team-level processes, which is another reason most of Big Tech does not push heavyweight team processes. This is not to say these organizations don't have challenges with productivity, but most of these challenges are not ones that a heavyweight process would solve. Challenges that these companies are working on include:

- **Developer productivity.** How can engineers spend more time writing code that moves the needle for the goals of their team, instead of being bogged down by infrastructure issues or working around problems with their dependencies? Platform teams is an approach that helps, but there is a lot more to unpack here. We'll cover more about this in future newsletters.
- **Repaying tech and architecture debt.** All of Big Tech moves fast and responds to new opportunities quickly. In doing so, companies often take shortcuts that result in tech and architecture debt. How can repaying this debt in a sensible way be part of the everyday process, instead of having "big bang" tech debt payoff projects?
- **Team structures that match organizational goals.** The goals of the company, and sub-organizations change regularly. How can the team structure reflect these changes, while minimizing disruption to team cohesion?
- **Slack time for innovation and unexpected work.** For teams that are expected to innovate, how do you create slack time to make this happen?
- **Keeping up the pace as the engineering team grows.** The more engineers a company has, the more overhead it takes to communicate, or make decisions impacting most engineers. How can organizations keep moving just as fast after they double in size? What are the processes and structure choices that allow teams to stay nimble and move fast, regardless of organizational size?
- **Durable excellence.** How can the whole organization improve its throughput, while engineers also stay happy, and improvements stick around long enough to compound? The phrase 'durable excellence' is from the article [Staying on the path to high performing teams](#) by Will Larson.

In Defense of Scrum

Should companies dismiss Scrum and other methodologies just because most of Big Tech has done so? This would be a mistake.

There are many contexts in which switching to Scrum makes perfect sense and will result in better productivity. Skype was an example where this change did help the company, and Whatsapp would have likely won the consumer calling space, regardless of what methodology Skype used.

A few situations where Scrum can be a good alternative:

1. "Kitchen sink teams" which have everything thrown at them, typically find that managing stakeholders with a heavyweight process like Scrum is a win. Stakeholders are educated to understand that an ongoing sprint cannot be interrupted and that new feature requests need to be groomed. Teams with conflicting priorities get to execute with fewer interruptions, thanks to the sprint structure giving space for the team to ignore these interruptions.

"Kitchen sink teams" are typical within non-tech companies, where the business has no understanding of how engineering works. Scrum helps rein in the stakeholders and educates them on software development processes, while giving the engineering team breathing room to execute. They are also common in early-stage startups, where there is one engineering team to build everything.

"Kitchen sink teams" occasionally appear in Big Tech, when a product team gets too many responsibilities. In all cases, moving over to a process that gives the team space to breathe, like Scrum, makes perfect sense. However, as teams are autonomous and empowered, more often than not they instead update their team charter, so team members can immediately say no to work the team does not own.

2. "Forming" and "storming" stages of new teams. Psychologist Bruce Tuckman came up with the phrase "forming, storming, norming, and performing" as phases on how groups work. This model very much fits how most engineering teams evolve as well.

When a new team is assembled, that team needs to decide how it will work. Reaching for an off-the-shelf approach like Scrum is almost always a better choice to start with,

than group members who are unfamiliar with each other coming up with custom processes – or none at all. Going with a well-documented approach like Scrum can also be useful if team members have conflicting, non-compatible opinions on “the right way to work.”

The nice thing about Scrum is how retrospectives are part of the process. This allows teams to reflect on their ways of working. Over time, teams with autonomy to change their working style usually end up dropping heavyweight Scrum rules they don’t need and develop a custom working style.

3. Speeding up shipping to once every few weeks, from a cadence less frequent than this. Scrum, together with weeks-long sprints can help teams move to more frequent shipping, so long as this frequency is not above the sprint length. For many non digital-first organizations, this type of acceleration is a big win.

Speeding up shipping was one of the major reasons Skype moved to Scrum in 2012. Before the transition, the teams shipped once every few months. After the transition, each team shipped once or twice a month. Note that the teams that ended up shipping more frequently than this were ones which decided to drop Scrum, as the process makes little sense with short sprint lengths.

4. Companies where uniformized project progress reporting is considered a must-have. Scrum and JIRA tend to go hand-in-hand, and there is no better tool for org-level reporting than JIRA. I have observed many organizations introducing Scrum with the expectation that leadership teams would get more visibility into teams, and be able to pinpoint teams that need help.

Unified reporting and drilling down to team-level priorities was one of the benefits Skype’s leadership also saw with the move to Scrum. Chris Matts, who was an agile coach at Skype at the time, [shares](#) how they implemented a *Strawberry-Jam-O-Meter* and a *Wrong-Order-O-Meter*, which would have been difficult to do, had not all teams used Scrum and JIRA:

“The Strawberry-Jam-O-Meter was used to identify teams that had the most backlog items in progress. Inspired by Jerry Weinberg’s “Law of Strawberry Jam”.

The Wrong-Order-O-Meter was used to identify teams that were working on items in the wrong order, according to the organization-wide backlog that the product owner organisation had specified."

My personal view is that in organizations with empowered teams, objectives and key results (OKRs), key performance indicators (KPIs) and goals are far better tools for aligning teams, than rolling out a rigid methodology like Scrum for the sake of reporting. However, in organizations that are not empowered, where teams and individuals are not autonomous, Scrum might work better than alternatives.

How Should You Run Teams?

We've seen how companies at various stages tend to run projects with different approaches, and how Big Tech generally does not mandate a single approach, though big firms have lots of organizational support to make this process work.

How you run teams should depend on your context. Relevant factors include your organizational structure, the people you work with, the autonomy and skills of those people, your competition, whether you're operating in "wartime" or "peacetime." The list goes on.

I'll leave you with the following ideas as food for thoughts.

- **Iterative changes always work better than 'big bang' ones.** A European tech company struggling with shipping very slowly hired a new VP of Engineering. This person decided to move the whole organization to a [NoEstimates](#) method in the first few months of their tenure. They organized a major event, hired a rock band, and unveiled the new way of working. The following weeks and months were chaos, and the organization reverted to doing what it did beforehand.
- **It's more work to teach someone to fish, than it is to catch a fish for them.** My approach to project management has been to coach and mentor members of my team to become project leads themselves. It was a lot more work upfront, but resulted in the team delivering more, people growing faster, getting promoted faster, and those people becoming engineering leaders faster than their peers.

This approach was one of my best decisions in an empowered environment. *Subscribers can access [my !\[\]\(2bdfe261b986065ee0ac76460d6528c9_img.jpg\) blueprint document](#) as well.*

- **Directing, mentoring and coaching all have their uses.** Directing – telling people exactly how to do something – is micromanaging when they can do it themselves. However, it's a supportive activity when they can't. Choose your approaches depending on whether you direct, mentor or coach and give space to people or teams, based on their capabilities as well. Over time, you should be doing little to no directing. But you might need to start with this.
- **The fewer people you need to make decisions, the faster you can make them.** If an engineer only needs to talk to an engineer to decide, that decision will be faster than if the engineer needs to talk to their project manager, who talks to another project manager, who talks to an engineer, who talks to... you get it.
- **Optimizing for reporting is optimizing for a low-trust environment.** Reporting at the executive levels is important. However, if you roll out project management methodologies that add heavy processes for the sake of reporting, then you'll get more process, lower trust, and people gaming whatever reports you're trying to produce.
- **Consultants will be biased to deliver easy-to-measure results because this is the simplest way to prove their value.** If the easy-to-measure result is a good goal, this makes consultants a good investment. Just make sure it is a worthwhile goal, and directionally correct.
- **Learning from direct competitors is underrated.** Understanding what a faster-moving competitor is doing – and experimenting with something similar – is a very smart one. Having a coffee with a peer at a competitor can be a great professional, and networking investment, not to mention one that may inspire you.
- **Some of the best engineers would rather quit than be micromanaged,** especially [when the job market is hot](#), and it's so easy to switch jobs. A relevant quote from a response to my survey: *"Recently, C-level executives have started to mandate the ways of working for all teams (everyone needs to follow the same methodology). It resulted in a lot of engineers leaving."*

I recommend getting inspiration from others, experiment, iterate, and move towards a high-trust environment where people are motivated. This is what I did, and how I [put a](#)

[structure in place](#) that helped every person on the team grow, for the benefit of everyone: the team, the company, and myself.

Up next: in next week's  subscriber-only issue  we'll cover advice to prepare for promotions with months to spare, and why you should start early.

Thanks to [Adriaan](#), [Alexandra](#), [Alex](#), [David](#), and [Steve](#) for their help in reviewing this article.

Read this post on [The Pragmatic Engineer blog as well](#).



279 Likes · 3 Restacks

5 Comments



Write a comment...



Soam Acharya Writes Soam's Substack Jan 25 · edited Jan 25

Hey Gergely - recent subscriber, now catching up on these articles. Great post that accurately summarizes my own personal experience. One small note - in the "In defense of Scrum" section, you write:

"Kitchen sink teams" occasionally appear in Big Tech, when a product team gets too many responsibilities."

It's been my observation that there are some platform teams even in Big(gish) Tech that will adopt Scrum precisely because they do have a variety of requests thrown at them. For example, some Security and Infra teams come to mind. They service engineering requests from all over the company (eg. create an S3 bucket, associate policies with that bucket, create a new VPC and so on) and hence must adopt Scrum to convey a notion of capacity and priorities.

 LIKE (2)  REPLY ...



Leponzo May 24, 2022

"Teams rotated Scrum Master roles"

Isn't that a bad practice?

♡ LIKE (1) 💬 REPLY ...

2 replies by Gergely Orosz and others

3 more comments...

© 2023 Gergely Orosz · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great writing